

FocusMaxxer — UI Internals: Non-Obvious Widgets & Patterns

A mechanics-level companion to [ARCHITECTURE.md](#). That document explains *what the UI shows and why* (domain mapping, screen map, data flow). This one explains *how the trickier pieces of Flutter machinery underneath it actually work* — the widgets, mixins and APIs that are easy to misread, easy to misuse, or easy to copy-paste incorrectly into new code. It deliberately does **not** describe widget trees screen-by-screen.

How to read this document. Sections are ordered **most-critical-first**: Section 1 covers the things that can silently corrupt state, leak listeners, or crash on a disposed object; Section 6 covers cosmetic idioms that are safe to get wrong. Within each section, the same ordering applies. Every claim is anchored to a `file.dart:line` reference — if the code has since moved, the reference is stale and the source wins.

Scope. All of `lib/` was swept for uncommon-Flutter-API usage (grep pass on ~35 symbol patterns), and every file that matched (23 of 36) was read in full. The patterns below are grouped by *mechanism*, not by file, because the same mechanism (e.g. the listener-capture-and-dispose idiom) is intentionally repeated with small variations across several screens — seeing them side by side is the point.

Quick-reference index

#	Pattern	Criticality	Where (primary)
1.1	<code>WidgetsBindingObserver</code> tri-state lifecycle (<code>inactive</code> / <code>paused</code> / <code>detached</code>)	 Critical	<code>clock_provider.dart</code> , <code>cognitive_engine_provider.dart</code> , <code>analytics_provider.dart</code>
1.2	Listener-capture-then-dispose idiom for cross- <code>ChangeNotifier</code> subscriptions	 Critical	<code>focus_mode_page.dart</code> , <code>break_mode_page.dart</code>
1.3	Re-entrancy guards around async state transitions	 Critical	<code>cognitive_engine_provider.dart</code>
1.4	<code>notifyListeners()</code> -after- <code>dispose()</code> guard	 Critical	<code>cognitive_engine_provider.dart</code>
1.5	<code>PopScope</code> : hard block vs. conditional callback	 High	<code>focus_mode_page.dart</code> , <code>break_mode_page.dart</code> , <code>profile_page.dart</code> , <code>session_report.dart</code>
1.6	<code>GlobalKey<FormState></code>	 Medium	<code>profile_page.dart</code>

#	Pattern	Criticality	Where (primary)
2.1	<code>context.watch</code> / <code>.read</code> / <code>.select</code> / <code>Consumer</code> — four ways to read a Provider	● Critical	<code>home_tab.dart</code> , throughout
2.2	<code>ProxyProvider</code> / <code>ChangeNotifierProxyProvider</code> tiered DI graph	● High	<code>main.dart</code>
2.3	Virtual-clock tick + catch-up loop	● Critical	<code>clock_provider.dart</code> , <code>cognitive_engine_provider.dart</code>
3.1	No <code>AnimationController</code> anywhere — and why	● Medium	(absence, app-wide)
3.2	<code>TweenAnimationBuilder</code> — implicit re-animation on rebuild	● Medium	<code>biometric_ring.dart</code> , <code>home_tab.dart</code> , <code>login_page.dart</code> , <code>onboarding_slide.dart</code>
3.3	<code>AnimatedOpacity</code> + <code>AnimatedSlide</code> combos (fractional-offset gotcha)	● Medium	<code>break_mode_page.dart</code> , <code>onboarding_slide.dart</code>
3.4	<code>AnimatedContainer</code> / <code>AnimatedPositioned</code> for the glow crossfade	● Low	<code>break_mode_page.dart</code> , <code>onboarding_page.dart</code>
3.5	<code>PageRouteBuilder</code> -driven transitions (no manual controller)	● Medium	<code>route_transitions.dart</code>
4.1	<code>CustomScrollView</code> + Sliver family	● High	<code>profile_page.dart</code> , <code>home_tab.dart</code> , <code>analytics_tab.dart</code> , <code>session_report.dart</code>
4.2	<code>SliverChildListDelegate</code> vs. <code>SliverChildBuilderDelegate</code>	● Medium	<code>profile_page.dart</code> vs. <code>analytics_tab.dart</code>
4.3	<code>PageView</code> / <code>PageController</code> : locked shell vs. swipeable carousel + <code>PageStorageKey</code>	● Medium	<code>home_dashboard.dart</code> , <code>onboarding_page.dart</code>
5.1	<code>BackdropFilter</code> frosted-glass recipe	● Medium	6+ files
5.2	<code>InkWell</code> needs a <code>Material</code> ancestor	● High	<code>analytics_tab.dart</code>
5.3	<code>GestureDetector</code> + <code>FocusScope.unfocus()</code> keyboard dismissal	● Low	<code>login_page.dart</code> , <code>profile_page.dart</code>
5.4	<code>IgnorePointer</code> for decorative layers	● Low	<code>ambient_glow.dart</code>
5.5	<code>fl_chart</code> : <code>LineChart</code> , <code>LineTouchData</code> , <code>VerticalRangeAnnotation</code>	● Medium	<code>session_report.dart</code>
6.1	<code>HapticFeedback</code> severity vocabulary	● Low	app-wide
6.2	<code>FontFeature.tabularFigures()</code>	● Low	<code>biometric_ring.dart</code> , <code>focus_mode_page.dart</code>

#	Pattern	Criticality	Where (primary)
6.3	Dart records + destructuring in <code>context.select</code> / return types	● Low	<code>home_tab.dart</code> , <code>onboarding_page.dart</code>
6.4	Lookbehind regex for camelCase → Title Case	● Low	<code>settings_components.dart</code>
6.5	<code>WidgetStateProperty.resolveWith</code> (Material 3 theming)	● Low	<code>app_theme.dart</code>

1. Lifecycle & concurrency correctness — ● Critical

This is the highest-risk section: every pattern here, done wrong, produces a bug that **only reproduces on a real device under specific timing** (backgrounding, a slow DB write, a widget torn down mid-navigation) — exactly the failure mode this app's `flutter analyze` -only, no-emulator workflow is least likely to catch by chance (see `CLAUDE.md` 's verification workflow).

1.1 `WidgetsBindingObserver` and the three-state lifecycle trap

Three classes mix in `WidgetsBindingObserver` and override `didChangeAppLifecycleState` : `GlobalClockProvider` (`clock_provider.dart:13,104-125`), `CognitiveEngineProvider` (`cognitive_engine_provider.dart:44-45,207-234`), and `AnalyticsProvider` (`analytics_provider.dart:14,56-61`).

The trap: `AppLifecycleState` has more than "foreground/background" — it has `resumed` , `inactive` , `paused` , `detached` (and `hidden` on newer engines). `inactive` fires for things that *look* like backgrounding but aren't: the notification shade, the app switcher, a system permission dialog. The app is still visible and running.

- `GlobalClockProvider` only stops its `Timer.periodic` on `paused` and only fast-forwards on `resumed` — it deliberately ignores `inactive` , otherwise pulling down the notification shade during a 4-hour focus session would freeze virtual time and silently distort the whole SAFTE/stress simulation.
- `CognitiveEngineProvider.didChangeAppLifecycleState` has an explicit comment (`cognitive_engine_provider.dart:210-216`) spelling this out: only `paused` arms the AFK/anomaly overlay, and it arms it for **focus, calibration, and break alike**, because the recovery decision depends on the same continuous HR stream the break phase needs too.
- `AnalyticsProvider` reacts to the same `paused` transition for a third, narrower reason: `didChangeAppLifecycleState` (`analytics_provider.dart:56-61`) calls `saveWorkloadToDisk()` to flush the daily-worked-seconds counter to `SharedPreferences` — the same "commit before the process dies" concern as the other two providers, just persisting a plain counter instead of a DB write or an armed overlay.
- `detached` is handled separately and is **not** the mirror of `paused` : it triggers `_commitSessionIfValid()` as a *fire-and-forget best effort* (`cognitive_engine_provider.dart:227-233`) — on modern Android the OS does not actually

await this callback before killing the process, so this is a best-effort save, not a guarantee. Anyone tempted to `await` more work here should know it may never run to completion.

If you copy this pattern: never react to `inactive` as if it were backgrounding, and always re-check `paused / resumed` pairing (a `paused` without a later `resumed` — e.g. process death — must leave no dangling timers; that's what `dispose()` is for, see 1.2).

1.2 Listener-capture-then-dispose: two idioms, same bug avoided

`FocusModePage` and `BreakModePage` both need to react when `CognitiveEngineProvider` finishes a session, so they attach a raw `addListener` (not `context.watch`, which would be an unconditional rebuild subscription) and must remove it in `dispose()`. Removing it requires **holding a reference to the exact provider instance**, because by the time `dispose()` runs, `context` may no longer be safely readable via `Provider.of / context.read`.

Two different idioms solve this, worth contrasting:

- `FocusModePage` uses `didChangeDependencies` with a null-aware capture-once guard: `dart _engineListener ??= context.read<CognitiveEngineProvider>() ..addListener(_onStateChange); (focus_mode_page.dart:45-49)`. `didChangeDependencies` can run more than once (e.g. on `InheritedWidget` changes), so the `??=` guard is what prevents a duplicate subscription.
- `BreakModePage` instead subscribes inside a **post-frame callback** from `initState`: `dart WidgetsBinding.instance.addPostFrameCallback(_) { if (!mounted) return; final engine = context.read<CognitiveEngineProvider>(); _engineRef = engine; engine.addListener(_checkAutoRoute); };` (`break_mode_page.dart:56-66`). `initState` cannot safely call `context.read` for an `InheritedWidget`-based provider in all cases and, more importantly, the widget can be disposed *before the callback ever runs* (e.g. a rapid double-navigation) — which is exactly why the captured reference field (`_engineRef`) is **nullable**: it is what lets `dispose()` no-op safely (`_engineRef?.removeListener( ... )`) instead of throwing on a null.

Both converge on the same invariant: **the field that `dispose()` reads must be populated by the same code path that calls `addListener`, and must be null-safe**, because `dispose()` can run before that path executes.

1.3 Re-entrancy guards around async state transitions

Several places in `CognitiveEngineProvider` and `BreakModePage` protect against the same class of bug: **an async operation (DB write, navigation) is in flight, and a second trigger fires before it resolves**.

- `_isNavigating` in `BreakModePage` (`break_mode_page.dart:48,84,104,351-352`) guards against `_checkAutoRoute` (an engine listener) and the manual END long-press racing each other into a double `Navigator.push`.
- `_isCommitting / _sessionCommitted` in `CognitiveEngineProvider` (`cognitive_engine_provider.dart:103-104,562-580`) make `_commitSessionIfValid()` idempotent: a

race between `endSession()` and the `detached` -lifecycle fire-and-forget commit (1.1) must persist the session **at most once**, because `AnalyticsProvider.commitValidatedSession` is deliberately *not* idempotent — it increments the daily-worked counter on every call ([analytics_provider.dart:75-76](#)). Without the guard, a race would double-count a session's minutes toward the daily cap.

- `endSession()` sets `_currentState = EngineState.sessionEnded` **synchronously, before** the `await _commitSessionIfValid()` ([cognitive_engine_provider.dart:639-652](#)): a tick that fires while the DB write is still pending sees the terminal state first and cannot re-enter the segment logic and call `endSession` a second time.
- `FocusModePage._onStateChange` ([focus_mode_page.dart:58-63](#)) solves the same class of problem with a **different shape**: instead of a boolean flag, it calls `_engineListener!.removeListener(_onStateChange)` as the very first line once it observes `sessionEnded` — its own comment says this is "so we never fire a second navigation mid-transition." Unsubscribing is itself the guard here, since the listener can't fire a second time if it's no longer attached; worth recognizing as a valid alternative to a boolean flag when the re-entrant call is a listener callback rather than an independently-triggerable method.

Takeaway for new async engine methods: set the terminal/guard flag *before* the first `await`, not after — the whole point is to close the re-entrancy window synchronously.

1.4 `notifyListeners()` after `dispose()` — the disposed-ChangeNotifier crash

`ChangeNotifier.notifyListeners()` throws if called after `dispose()`. Because `CognitiveEngineProvider.endSession()` and `_triggerDailyLimit()` both `await` a DB write before notifying, the provider can be disposed **during** that `await` (e.g. the user backgrounds and the OS reclaims the widget tree). Both methods guard the post-`await` `notifyListeners()` with an explicit `_isDisposed` flag set in `dispose()`:

```
if (_isDisposed) return;
notifyListeners();
```

([cognitive_engine_provider.dart:648-651,660-667](#)). `_triggerDailyLimit()` additionally re-checks `_isDisposed` inside a `Future.delayed` callback scheduled from within itself — a second, independent disposal window that is easy to forget when only guarding the first `await`.

Asymmetry worth knowing about: `AnalyticsProvider` has the structurally identical shape — `await` a DB/prefs write, then `notifyListeners()` — in `commitValidatedSession`, `deleteSession`, `resetDailyWork`, and `_loadSessionsHistory` ([analytics_provider.dart:77-132](#)), but has **no** `_isDisposed` flag anywhere. This hasn't surfaced as a bug because `main.dart` wires it via `ChangeNotifierProxyProvider` with `previous ?? AnalyticsProvider(...)` ([main.dart:116-121](#)), making it a de-facto root-lifetime singleton that's never disposed during normal app life — but if this provider is ever re-scoped narrower (e.g. behind a route-local `MultiProvider`), it would need the same guard as `CognitiveEngineProvider`.

1.5 `PopScope`: hard block vs. conditional-with-callback

`PopScope` (the modern, non-deprecated replacement for `WillPopScope`) appears in four places, in two distinct shapes:

- **Hard block**, `canPop: false` with no callback — `FocusModePage` ([focus_mode_page.dart:95-98](#)) and `BreakModePage` ([break_mode_page.dart:136-139](#)). The system back gesture is completely swallowed; the *only* way out is the explicit BREAK/RESUME/END controls, so a session can never be abandoned by an accidental back-swipe without going through the commit path in 1.3.
- **Conditional, with `onPopInvokedWithResult`** — `ProfilePage` (`canPop: !_hasUnsavedChanges`, [profile_page.dart:246-258](#)) and `SessionReportPage` (`canPop: isHistory`, [session_report.dart:79-82](#)). Here the pop is allowed to proceed silently when the condition is already true; when it's false, the pop is intercepted (`didPop` is `false` in the callback) and a confirmation dialog decides whether to pop manually afterwards.

Non-obvious bit: `onPopInvokedWithResult(didPop, result)` is called *after* Flutter has already decided whether the pop happened. If `didPop` is `true` the callback should just return (as `ProfilePage` does) — trying to act as if the pop hadn't happened yet is the classic misuse of this API left over from `WillPopScope` muscle memory, where the equivalent callback ran *before* the pop and could still veto it via a returned `bool`.

1.6 `GlobalKey<FormState>`

The only `GlobalKey` in the codebase: `final _formKey = GlobalKey<FormState>();` in `ProfilePage` ([profile_page.dart:29](#)), attached to the single `Form` wrapping the three `SettingsTextField`s ([profile_page.dart:372-401](#)). It exists purely to call `_formKey.currentState!.validate()` in `_saveProfile()` ([profile_page.dart:77](#)) without threading a `FormState` callback through the widget tree. Since it is the only `GlobalKey`, there is no risk of the classic "same `GlobalKey` used twice in one frame" crash here — but it's worth knowing this is the *only* legitimate reason a `GlobalKey` exists in this codebase, should another one turn up in a diff.

2. State propagation & Provider performance — Critical / High

2.1 `context.watch` / `.read` / `.select` / `Consumer<T>` — pick deliberately

All four idioms for reading a `provider` package value appear in this codebase, and the choice is **not interchangeable**:

Idiom	Rebuilds on	Used for
<code>context.read<T>()</code>	never (one-shot read)	event handlers, <code>initState</code> , one-time snapshots

Idiom	Rebuilds on	Used for
<code>context.watch<T>()</code>	every <code>notifyListeners()</code> on <code>T</code>	when the whole subtree should track <code>T</code> live
<code>context.select<T, R>(selector)</code>	only when the <i>selected</i> <code>R</code> changes (<code>==</code>)	throttling rebuilds to a derived value
<code>Consumer<T></code>	every <code>notifyListeners()</code> on <code>T</code> , scoped to its <code>builder</code>	same as <code>watch</code> , but confines the rebuild to a sub-widget instead of the whole <code>build()</code>

The performance-critical case is `HomeTab`: the SAFTE effectiveness score updates on **every virtual clock tick (~83ms real-time at the 60× default)**, but the UI only needs to redraw when the *rounded, displayed* value changes. `_ReadinessCard` selects the floored score, not the raw double:

```
final double score = context.select<GlobalClockProvider, double>(
  (clock) => safte.getStateAt(clock.currentTime).effectiveness.floorToDouble(),
);
```

([home_tab.dart:175-178](#)), and `_KeyFactorsSection` goes further, selecting a whole **record** of bucketed status strings/colors so the tiles only rebuild when a *label* actually flips ([home_tab.dart:311-333](#)). Without this, every tile in the Home tab would rebuild dozens of times per second.

`HomeTab.build` itself uses `Consumer<AuthProvider>` ([home_tab.dart:39-56](#)) rather than `context.watch`, per its own comment, specifically so the greeting rebuilds once the nickname resolves after login — functionally close to `watch` here, but scoping the subscription to the `Consumer`'s builder keeps the intent explicit at the call site.

Rule of thumb when adding new UI against the engine/SAFTE providers: if the underlying value changes every tick but the on-screen representation is coarser (a percentage, a bucketed label, a boolean threshold), reach for `context.select` before `context.watch` — this app treats it as the default performance safeguard, not an optimization to bolt on later.

2.2 `ProxyProvider` / `ChangeNotifierProxyProvider`: the tiered DI graph

`main.dart`'s `MultiProvider` list is explicitly commented as **ordered by dependency tier** — "each tier may only read providers declared above it" ([main.dart:83-84](#)). Two entries use the `Proxy` variants specifically to let a provider **depend on another provider without a `BuildContext` lookup at construction time**:

- `ProxyProvider<AuthProvider, ImpactApiService>` ([main.dart:96-105](#)) wires `ImpactApiService.onSessionExpired` to call `auth.logout()` — a callback the service invokes on its own when the server rejects a refresh token. The `update` callback reuses the `previous` instance (`previous ??= ...`) rather than constructing a new `ImpactApiService` on every rebuild, which would otherwise silently drop in-flight requests/tokens.
- `ChangeNotifierProxyProvider<SessionRepository, AnalyticsProvider>` ([main.dart:116-121](#)) follows the same reuse-via-`previous` pattern.

Why this matters for correctness, not just style: if a future provider is inserted in the wrong tier (e.g. a Tier 2 provider trying to `context.read` a Tier 3 one inside its own `create`), the app crashes at startup with a `ProviderNotFoundException` — the ordering comment is the only guard against this, since Dart's type system cannot express "must be declared after X".

2.3 The virtual-clock tick + catch-up loop

This is the single most intricate piece of state machinery in the UI-adjacent layer, spanning two providers:

- `GlobalClockProvider` runs a `Timer.periodic` whose **real** interval is the virtual tick length divided by the speed multiplier — a 5 s virtual tick at 60× fires roughly every 83 ms (`clock_provider.dart:72-92`). Changing the speed multiplier at runtime (the profile's dev-tools dropdown, `settings_components.dart:289-356`) cancels and rebuilds this timer at the new interval, but **each notify still advances virtual time by exactly one tick** — so the listener downstream never has to special-case a speed change mid-session.
- `CognitiveEngineProvider._onGlobalTick` (`cognitive_engine_provider.dart:264-319`) does **not** trust that it gets called once per virtual tick. It diffs its own `_internalClock` against `clock.currentTime` and replays as many `_processTick()` calls as the delta implies — this is what makes the app survive a dropped frame or a fast-forwarded resume after backgrounding (1.1) without losing simulated seconds.
- The replay is capped at `maxCatchupTicks = 360` (30 virtual minutes) (`cognitive_engine_provider.dart:79`). Beyond that, replaying every tick would freeze the UI thread, so the engine instead **voids the session** (or, during calibration, defers to the already-armed anomaly overlay) rather than silently fast-forwarding through half an hour of fake biometrics.

If you're debugging "the ring/timer seems to jump": check whether the observed jump is a legitimate catch-up (a real gap occurred) or a symptom of the `_internalClock / clock.currentTime` diff going out of sync — the two clocks are deliberately kept as separate variables, and `_onGlobalTick` is the **only** place that diffs them and replays a catch-up. (`_internalClock` is also directly re-synced to `clock.currentTime` elsewhere — the constructor, `startSession()`, and `resetEngine()` — but those are resets, not catch-up logic; if a jump doesn't line up with one of those three call sites, the bug is almost certainly inside `_onGlobalTick` itself.)

3. Animation system — Medium

3.1 There is no `AnimationController` in this codebase — and that's deliberate

A project this animation-heavy (glows, rings, slide-ins, fades, route transitions) would typically reach for `AnimationController` + `SingleTickerProviderStateMixin`. **Neither appears anywhere in `lib/`.** Every animation is one of:

1. An **implicit animation widget** (`TweenAnimationBuilder`, `AnimatedOpacity`, `AnimatedSlide`, `AnimatedContainer`, `AnimatedPositioned`) — Flutter owns the controller internally and drives it whenever the target value changes across a rebuild (3.2–3.4).
2. The **animation object Flutter's Navigator already provides** to a `PageRouteBuilder.transitionsBuilder` — the three custom routes in `route_transitions.dart` only *consume* this animation (via `.drive( ... )`), they never construct one (3.5).

The practical consequence: nothing in the UI layer needs a `TickerProvider` mixin, `vsync:`, or manual `.dispose()` of an animation object — one entire class of leak (a forgotten `controller.dispose()`) is structurally absent. The trade-off is that anything requiring true *continuous, self-driving* animation (an idle pulsing/breathing loop that runs regardless of state changes) is not expressible with these widgets — there is currently no such animation in the app (the `BiometricRing` and `_RecoveryRing` are both static/reactive-only; see the note in the Appendix about `ARCHITECTURE.md` drift on this exact point).

3.2 `TweenAnimationBuilder`: implicit re-animation on rebuild

`TweenAnimationBuilder<T>` re-runs its animation **every time the widget rebuilds with a different `tween.end`**, interpolating from wherever the *previous* animation last stopped — no controller, no explicit `forward()`. Four call sites:

- `BiometricRing` animates the ring's fill percentage on every tick: `Tween<double>(end: progressPercentage)` over 400 ms (`biometric_ring.dart:93-95`). Each new `progressPercentage` (a discrete, tick-driven value) is smoothed into a continuous-looking sweep purely because the builder animates *toward* it every rebuild.
- `_ReadinessCard` in the Home tab animates the readiness arc the same way, but over a much longer 1800 ms with `Curves.easeOutCubic` (`home_tab.dart:239-243`) — a deliberately slower reveal since this score changes far less often (2.1).
- `LoginPage._buildAnimatedForm` uses a `Tween(begin: 0.0, end: 1.0)` purely as a **one-shot mount animation** (opacity + a computed vertical offset, `Offset(0, 20 * (1 - opacity))`) (`login_page.dart:182-195`) — since the widget is built once and `end` never changes again, this degrades to a simple "animate in on first frame" with no controller needed.
- `OnboardingSlide._buildAnimatedIcon` animates a **scale factor** that switches between two fixed endpoints based on `isActive` (`onboarding_slide.dart:60-89`): every time the carousel page changes, the previously-active slide's icon tween reverses toward the inactive scale while the newly-active one animates toward the active scale — both driven by the same parent `setState` in `OnboardingPage._buildPageView`.

Gotcha to flag in review: `TweenAnimationBuilder`'s `builder` receives the animation's live value on every frame — any expensive computation placed directly inside `builder` (rather than passed via the `child` parameter) reruns every frame. None of the four call sites above currently do this, but it's the first thing to check if a new one is added and the ring stutters.

3.3 `AnimatedOpacity` + `AnimatedSlide` combos: matched durations, fractional offsets

Two places fade **and** slide a widget in together, using two separate implicit-animation widgets with the same duration so they read as one motion: the break screen's "LOOK AWAY" card ([break_mode_page.dart:256-263](#)) and the onboarding slide's text block ([onboarding_slide.dart:94-101](#)).

Non-obvious unit: `AnimatedSlide.offset` is a **fraction of the child's own size**, not logical pixels — `Offset(0, 0.2)` means "20% of this widget's height," not "20 logical pixels." Both call sites rely on this to make the "rest lower, rise into place" effect independent of the actual widget's measured height. Reading `0.2` as pixels (an easy mistake coming from `Transform.translate`, which is pixel-based — see 3.2's login form, which uses `Transform.translate` precisely because it needed a pixel offset) would produce a barely-perceptible 0.2px slide instead.

3.4 `AnimatedContainer` / `AnimatedPositioned` for the glow crossfade — and why it's *not* the shared `AmbientGlow`

Most screens paint a decorative background blob using the shared `AmbientGlow` widget (a plain, non-animated `RadialGradient`, [ambient_glow.dart](#)). Three screens (`BreakModePage`, `LoginPage`, `OnboardingPage`) deliberately hand-roll their own glow instead, and each one's source comment explains why:

- `BreakModePage` needs the glow's **color** to crossfade (teal ↔ amber) when the break enters the "extended/fatigue" state — only an `AnimatedContainer` with an animatable `decoration` gives that transition; a static `AmbientGlow` would snap ([break_mode_page.dart:34-39](#)).
- `OnboardingPage` needs the glow's **position** to drift between per-page corners as the carousel advances, wrapping the same `AnimatedContainer` in an `AnimatedPositioned` ([onboarding_page.dart:154-186](#)).
- `LoginPage` and `OnboardingPage` also need a **hard-edged circle diffused by a full-screen `BackdropFilter` blur** (5.1) rather than a gradient — a look `AmbientGlow`'s `RadialGradient` approach cannot reproduce.

This is a useful example of "**why does this look almost like the reused helper but isn't**" — always worth checking the local comment before "simplifying" one of these three to use `AmbientGlow`; the divergence is intentional, not leftover duplication.

3.5 `PageRouteBuilder` -driven transitions

`route_transitions.dart` defines three named route classes, each a thin `PageRouteBuilder` whose `transitionsBuilder` only *chains* the Navigator-provided `animation` — none of them creates its own `AnimationController`:

- `ImmersiveRoute` — fade + scale from 95%→100%, an `easeOutCubic` "dive in," used for entering Focus mode or opening a report (600 ms in / 400 ms out).
- `FadeRoute` — a plain cross-fade (400 ms/300 ms), used for Onboarding→Login and Dashboard→Profile.
- `SessionActiveRoute` — a very fast 250 ms fade, used only between the tightly-coupled Focus↔Break screens, so switching feels immediate rather than like a full navigation.

The `Tween<double>(begin: 0.95, end: 1.0).chain(curveTween)` idiom in `ImmersiveRoute` ([route_transitions.dart:23-25](#)) is the standard way to apply a curve to a tween without a controller: `.chain` composes two `Animatable`s, and `animation.drive(...)` evaluates the composed chain against the Navigator's own transition progress.

4. Scrolling & layout — High / Medium

4.1 `CustomScrollView` + the Sliver family

Four screens (`ProfilePage`, `HomeTab`, `AnalyticsTab`, `SessionReportPage`) use `CustomScrollView` instead of a plain `ListView` / `SingleChildScrollView`, specifically to mix a **collapsing app bar** with regular scrolling content in one scroll physics system. The recurring slivers:

- `SliverAppBar(pinned: true, stretch: true, ...)` — stays pinned at the top once scrolled past, and `stretch: true` lets an over-scroll (pulling down past the top) stretch the header instead of just showing empty space. Both `PremiumSliverAppBar` ([premium_sliver_app_bar.dart:42-84](#)) and `ProfilePage`'s inline `SliverAppBar` ([profile_page.dart:278-312](#)) pair this with `flexibleSpace: BackdropFilter(...)` (5.1) for the frosted-glass look, and `stretchModes: [StretchMode.zoomBackground]` (plus `fadeTitle` in the shared bar) to control what actually happens during that overscroll stretch.
- `SliverPadding` wrapping a `SliverList` / `SliverFillRemaining` — padding a sliver requires `SliverPadding`, not a plain `Padding` widget (which does not implement the sliver protocol and cannot be a direct child of `CustomScrollView`).
- `SliverFillRemaining(hasScrollBody: false)` — used for the Analytics empty state ([analytics_tab.dart:76-95](#)) and the whole report body ([session_report.dart:155-176](#)). `hasScrollBody: false` tells the sliver its child is a fixed-size, non-scrolling block that should simply fill (and center within) the remaining viewport — omitting this flag on a `Column`-based child is a common source of a `RenderFlex overflowed` or an unscrollable, clipped layout.
- `SliverToBoxAdapter` — the simplest sliver adapter, used only to insert a fixed-height spacer ([analytics_tab.dart:39](#)) so the last card clears the bottom navigation bar.

4.2 `SliverChildListDelegate` VS. `SliverChildBuilderDelegate`

Both delegates back a `SliverList`, but they differ in **when** children are built:

- `ProfilePage` uses `SliverChildListDelegate([ ... ])` ([profile_page.dart:317](#)) — an **eagerly built, fixed list** of widgets. Appropriate here because the settings page has a small, static number of sections; there's nothing to gain from lazy building.
- `AnalyticsTab` uses `SliverChildBuilderDelegate((context, index) => ..., childCount: sessions.length)` ([analytics_tab.dart:54-63](#)) — **lazily built on demand** as the user scrolls, which matters once the session history grows past what fits on screen (an unbounded, user-generated list is exactly the case `SliverChildListDelegate` would build eagerly and wastefully).

Rule of thumb: a fixed, small, known-at-build-time set of widgets → `SliverChildListDelegate` ;
anything backed by a variable-length data collection → `SliverChildBuilderDelegate` .

4.3 `PageView` / `PageController` : locked shell vs. swipeable carousel, and `PageStorageKey`

Two different `PageView` configurations, deliberately opposite:

- `HomeDashboard` pairs a `PageView` with `physics: const NeverScrollableScrollPhysics()` ([home_dashboard.dart:77-81](#)) — swiping is disabled entirely; the **only** way to switch tabs is the bottom `NavigationBar`, whose `onDestinationSelected` calls `_pageController.animateToPage( ... )` ([home_dashboard.dart:45-56](#)). This keeps horizontal swipe gestures free for content inside each tab (e.g. a future carousel) without fighting the tab-switch gesture.
- `OnboardingPage` instead uses a fully swipeable `PageView.builder` ([onboarding_page.dart:210-224](#)), since swiping between onboarding slides is the primary interaction, with the NEXT button as a secondary, `_pageController.nextPage( ... )`-driven path.

`HomeDashboard` 's two tab widgets are each given a `PageStorageKey` (`PageStorageKey('home_tab')` / `'analytics_tab'`, [home_dashboard.dart:27-30](#)) so that switching tabs and back preserves each `CustomScrollView` 's scroll offset — without the key, Flutter's default `PageStorage` bucket cannot distinguish the two scroll positions reliably once the widgets are recreated by the `PageView` .

5. Visual effects & interaction widgets —

Medium / Low

5.1 `BackdropFilter` frosted-glass recipe

The "frosted glass" look (blurred, semi-transparent panel over whatever scrolls or sits behind it) recurs in at least six places: `LoginPage` / `OnboardingPage` glow backgrounds, `SettingsGroup`, `PremiumSliverAppBar`, `OverlayScaffold` (AFK/calibration overlays), and the Analytics session card. The recipe is always the same three-layer stack:

```
ClipRect(                                // or ClipRRect for rounded corners
  child: BackdropFilter(
    filter: ImageFilter.blur(sigmaX: s, sigmaY: s),
    child: Container(color: tintColors.withAlpha(a), child: content),
  ),
)
```

Why the `Clip*` wrapper is not optional: `BackdropFilter` blurs everything painted behind it within the current layer's bounds, which — without a `ClipRect` / `ClipRRect` ancestor — can extend beyond the

widget's visual bounds and blur unrelated content, or blur a much larger backdrop than intended (rounded cards specifically need `ClipRRect` so the blur itself is clipped to the rounded shape, not just the tint color — see `SettingsGroup`, [settings_components.dart:38-44](#), and the Analytics card, [analytics_tab.dart:132-136](#)).

`OverlayScaffold` centralizes exactly this recipe for the two blocking modal overlays (AFK warning, calibration failure) so both share identical chrome and only vary blur strength / dim alpha ([afk_warning_overlay.dart:30-45](#)).

5.2 `InkWell` needs a `Material` ancestor — and why it's wrapped explicitly here

`InkWell`'s ripple/highlight paints onto the nearest `Material` ancestor. Inside a `BackdropFilter` + `ClipRRect` card (5.1), that ancestor is not guaranteed to be the right color/elevation, so the Analytics session card explicitly interposes a transparent `Material` between the blur and the `InkWell`:

```
child: Material(  
  color: Colors.transparent,  
  child: InkWell(onTap: ... , highlightColor: ... , splashColor: ... , ...),  
)
```

([analytics_tab.dart:136-138](#)). Without the explicit `Material(color: Colors.transparent)`, the ink splash would either paint onto a distant ancestor `Material` (visually detached from the card) or, in some nesting orders, silently fail to render at all. `SettingsActionRow`'s `InkWell` ([settings_components.dart:159-198](#)) gets away without an explicit wrap even though it sits inside `SettingsGroup`, which wraps its own `Column` in the exact same `ClipRRect` + `BackdropFilter` recipe as the Analytics card (5.1, [settings_components.dart:38-44](#)) — so "is there a `BackdropFilter` in between" is **not** actually what decides this: `Material.of(context)` walks the widget tree independently of intervening render-layer widgets like `BackdropFilter` / `ClipRRect`, so both `InkWell`s find the same kind of distant ancestor `Material` either way. The Analytics card's explicit local `Material(color: Colors.transparent)` is a defensive/scoping choice, not something `BackdropFilter` structurally forces — `SettingsActionRow` simply hasn't needed it in practice.

5.3 `GestureDetector` + `FocusScope.unfocus()` : dismiss the keyboard on background tap

`LoginPage` and `ProfilePage` both wrap their scrollable content in a `GestureDetector(onTap: () => FocusScope.of(context).unfocus())` so tapping anywhere outside a text field dismisses the keyboard. The two pick **different** `HitTestBehavior`:

- `LoginPage` uses `HitTestBehavior.opaque` ([login_page.dart:129-132](#)): the detector claims the tap even over empty space that has no visible content behind it (the `Stack` background is otherwise non-interactive).

- `ProfilePage` uses `HitTestBehavior.translucent` ([profile_page.dart:260-262](#)): taps are still detected for the unfocus side-effect, **but also continue on** to the `CustomScrollView` beneath — required here because `opaque` would otherwise compete with the scroll view's own gesture arena and could interfere with scrolling.

Rule of thumb: `opaque` when the `GestureDetector` sits over content that has no gestures of its own to preserve; `translucent` when a scrollable or otherwise gesture-sensitive widget sits underneath and must keep receiving its own gestures.

5.4 `IgnorePointer` for purely decorative layers

`AmbientGlow` wraps its `Container` in `IgnorePointer` ([ambient_glow.dart:52-53](#)) so the glow — despite being painted in a `Stack` on top of/behind interactive content depending on ordering — never intercepts a tap meant for the real UI beneath or around it. This is the correct tool specifically because the glow has *no* gesture handlers of its own to suppress; `AbsorbPointer` (which still consumes the hit-test but blocks it) is not needed since there's nothing to absorb.

5.5 `fl_chart`: the only chart in the app

`SessionReportPage._buildChart` ([session_report.dart:423-517](#)) is the sole `fl_chart` usage. Notable, easy-to-misread details:

- **Fixed Y-axis bounds** (`minY: 40`, `maxY: 160` bpm) regardless of the actual session's HR range — a deliberate choice so charts are visually comparable session-to-session and outlier spikes are clipped rather than rescaling the whole axis.
- **X axis is derived from the tick index**, not from stored timestamps: `startX = (i * tickDurationSeconds) / 60.0` converts a tick index to elapsed minutes. The tooltip then converts back the other way (`(spot.x * 60).round()`) to reuse the shared `formatHuman` seconds formatter — a two-way unit conversion worth tracing if the chart ever shows an off-by-one-tick time label.
- **`VerticalRangeAnnotation`** is used to *shade time bands* (focus vs. recovery), not to annotate a value range — one annotation per tick, each spanning `[startX, endX]`, tinted by the tick's recorded `state`. This is a slightly unusual use of an annotation API most examples show used sparingly for a handful of ranges, not once per data point.
- **`ExtraLinesData.horizontalLines`** draws the dashed average-HR reference line — a fixed horizontal line independent of the series data, easy to miss when scanning for how the average is visualized (it's not a second `LineChartBarData` series).
- **`LineTouchData.touchTooltipData.getTooltipItems`** must return one `LineTooltipItem` per *touched spot* (a list), even though this chart only ever has one series/one touched point at a time — a common point of confusion since the callback signature is built for potentially-overlapping multi-series tooltips.

6. Small idioms & conventions glossary — Low

These are safe to get "wrong" in the sense that nothing crashes — but inconsistent use would make the UI feel incoherent, so they're documented as conventions to follow, not just curiosities.

6.1 `HapticFeedback` severity vocabulary

The app uses four `HapticFeedback` calls as a consistent "weight" language, not decoration:

Call	Used for
<code>selectionClick()</code>	Minor, reversible UI selection (dropdown pick, tab switch, empty-submit rejection)
<code>lightImpact()</code>	A confirmed but low-stakes action (form submit, "long-press to end" hint)
<code>mediumImpact()</code>	A meaningful transition (onboarding finish, manual break start, live-debrief close)
<code>heavyImpact()</code>	An irreversible or high-stakes action (session end, daily-limit/AFK trigger, logout)

New interactive controls should pick from this table by *consequence*, not by what "feels right" locally — e.g. the END buttons in both Focus and Break use `lightImpact` for the tap-hint snackbar but `heavyImpact` for the long-press that actually ends the session ([focus_mode_page.dart:349,366](#), [break_mode_page.dart:335,353](#)) — the same control uses two different weights depending on which of its two gestures actually fired.

6.2 `FontFeature.tabularFigures()`

Applied to both live timers (`BiometricRing` 's center percentage is not tabular, but the session clocks are): `focus_mode_page.dart:555` and `break_mode_page.dart:520` . Tabular figures give every digit glyph the same advance width, so a running `m:ss` timer doesn't visibly jitter/reflow horizontally as digits change (e.g. `9:59` → `10:00` without this would shift width mid-second).

6.3 Dart records + destructuring

Two Dart-3-era record usages worth flagging for anyone less familiar with the feature:

- `_KeyFactorsSection` destructures a **positional record** directly out of `context.select`: `dart final (String reservoirStatus, Color reservoirColor, String circadianStatus, String inertiaStatus) = context.select<GlobalClockProvider, (String, Color, String, String)>( ... );` ([home_tab.dart:311-316](#)) — the selector's return type is an anonymous positional record, pattern-matched into four local variables in one declaration.
- `OnboardingPage._glowPositionFor` returns a **named-field record type** declared inline as the method's return type: `dart ({double top, double? right, double? left})` `_glowPositionFor(int page)` ([onboarding_page.dart:118](#)), replacing what the source comment calls out as an alternative "nested ternaries" approach — each field accessed by name (`pos.top` , `pos.right`) at the call site rather than by tuple position.

6.4 Lookbehind regex for camelCase → Title Case

`SimulatorSettingsRow._formatScenarioName` ([settings_components.dart:275-279](#)) turns an enum name like `partialRecovery` into `Partial Recovery` using `RegExp(r'(?<=[a-z])[A-Z]')` — a **zero-width positive lookbehind** that matches an uppercase letter only when immediately preceded by a lowercase one (i.e. every internal camelCase word boundary), then inserts a space before each match and title-cases the first letter. Worth knowing lookbehind exists in Dart's `RegExp` (it's a relatively recent addition to the engine) before reaching for a more verbose manual character-scan.

6.5 `WidgetStateProperty.resolveWith` (Material 3 theming)

`AppTheme.darkTheme`'s `navigationBarTheme` resolves icon/label color based on selection state via `WidgetStateProperty.resolveWith((states) => ...)` ([app_theme.dart:172-191](#)), checking `states.contains(WidgetState.selected)`. `WidgetState` / `WidgetStateProperty` are the Material-3-era rename of the older `MaterialState` / `MaterialStateProperty` API — functionally identical, but a codebase or Stack Overflow answer using the old names is not a typo, just a pre-rename source.

Appendix A — File → pattern quick lookup

File	Patterns documented here
<code>providers/clock_provider.dart</code>	1.1, 2.3
<code>providers/cognitive_engine_provider.dart</code>	1.1, 1.2 (consumed by), 1.3, 1.4, 2.3
<code>providers/analytics_provider.dart</code>	1.1, 1.3, 1.4
<code>providers/auth_provider.dart</code>	2.2 (consumer)
<code>providers/safte_provider.dart</code>	2.1 (data source), memoization (<code>_cachedTarget</code> / <code>_cachedState</code> , not separately sectioned — see <code>safte_provider.dart:47-51, 154-166</code>)
<code>main.dart</code>	2.2
<code>screens/focus_mode_page.dart</code>	1.2, 1.3, 1.5, 3.1, 5, 6.1, 6.2
<code>screens/break_mode_page.dart</code>	1.2, 1.3, 1.5, 3.3, 3.4, 6.1, 6.2
<code>screens/login_page.dart</code>	3.2, 3.4, 5.1, 5.3
<code>screens/onboarding_page.dart</code>	3.4, 4.3, 6.3
<code>screens/profile_page.dart</code>	1.5, 1.6, 4.1, 4.2, 5.1, 5.3
<code>screens/bootloader_screen.dart</code>	error-flow pattern (typed-exception routing, §1 sibling not separately sectioned — see <code>bootloader_screen.dart:84-97</code>)
<code>screens/session_report.dart</code>	1.5, 4.1, 5.5
<code>screens/home_dashboard.dart</code>	4.3

File	Patterns documented here
<code>screens/tabs/home_tab.dart</code>	2.1, 3.2, 4.1, 6.3
<code>screens/tabs/analytics_tab.dart</code>	4.1, 4.2, 5.1, 5.2
<code>utils/biometric_ring.dart</code>	3.2, 6.2
<code>utils/afk_warning_overlay.dart</code>	5.1
<code>utils/ambient_glow.dart</code>	5.4
<code>utils/onboarding_slide.dart</code>	3.2, 3.3
<code>utils/premium_sliver_app_bar.dart</code>	4.1, 5.1
<code>utils/route_transitions.dart</code>	3.5
<code>utils/settings_components.dart</code>	5.1, 5.2, 6.4
<code>utils/app_theme.dart</code>	6.5

Appendix B — Methodology & tooling notes

Approach. Rather than describing screens top-down, this document was built bottom-up from a codebase-wide sweep:

1. **Discovery** — a single `grep`-style regex pass across all of `lib/` (36 files) against ~35 symbol patterns named by the user as "example difficulty markers" (`AnimationController` , `TweenAnimationBuilder` , `Sliver*` , `InkWell` , `PopScope` , `GestureDetector` , `CustomScrollView` , `FocusScope` , `GlobalKey` , `LineTouchData` , plus ~25 adjacent Flutter APIs that tend to travel with the same class of subtlety: `WidgetsBinding*` , mixins, `Curved/Tween` machinery, `BackdropFilter` , `PageView` / `PageController` , etc.). 23 of 36 files matched.
2. **Full reads, no delegation** — every matched file was read in full (not excerpted) directly in this conversation, rather than farmed out to sub-agents. At ~6,300 lines across 23 files, the whole set fit comfortably in one context window; splitting the read across parallel agents would have fragmented the cross-file comparisons that make up most of this document's value (e.g. contrasting the two listener-capture idioms in §1.2, or the two `PopScope` shapes in §1.5) and would have required a second pass to reconcile terminology and criticality judgments across agents. For a codebase an order of magnitude larger, splitting by directory (`screens/` , `utils/` , `providers/`) across parallel read-only agents, each returning structured findings for a human/orchestrator pass to cross-reference, would be the more efficient split — the threshold is roughly "can one context window hold the whole matched set with room to reason about it," which this codebase is comfortably under.
3. **Classification** — each finding was tagged by *failure severity if misunderstood or miscopied* (crash/data-loss → ● , visible-but-recoverable UX bug → ● / ● , purely cosmetic → ●), not by how "advanced" the API looks. This is why, e.g., `PopScope` (●) outranks `fl_chart` 's touch tooltips (●) despite the latter looking more exotic: a `PopScope` mistake strands a session or discards unsaved data, while a chart tooltip mistake is at worst a display glitch.

4. **Cross-check against existing docs** — `docs/ARCHITECTURE.md` §11 already documents the UI at the domain-mapping level and was read first to avoid duplicating it. One drift was found worth flagging separately: §11.4–11.5 there describe the Focus ring and Break-mode ring as `CustomPainter`-based with a "breathing pacer" pulse and a `RepaintBoundary`; the current source (post the `simplified rings` commit in git history) implements both purely with `TweenAnimationBuilder` + `CircularProgressIndicator / Container` and has no pacer animation at all (see §3.1 and §3.2 above). `ARCHITECTURE.md` §11.4–11.5 should be refreshed to match; this document describes the current implementation.

Output format. Markdown is the source of truth (`UI_INTERNALS.md` , versioned in git, diffable); `UI_INTERNALS.pdf` is a generated, static render for quick sharing outside the repo (same toolchain already used for `ARCHITECTURE.pdf` : Python's `markdown` library to HTML, then a headless Chromium-based browser's `--print-to-pdf` to rasterize — no new software was installed to produce it). Regenerate the PDF after editing the Markdown; the Markdown, not the PDF, should be reviewed/diffed in future changes.